(i) $T(N) = 2T(N/2) + N$

$= 2\left[2T(N/4) + N/2\right] + N$

(ii) $T(N) = 4T(N/4) + 2N$

$= 4\left[2T(N/8) + N/4\right] + 2N$

$= 8T(N/8) + N + 2N$

(iii) $T(N) = 8T(N/8) + 3N.$

$T(N/2) \Rightarrow 2T(N/4) + N/2$

$T(N/4) \Rightarrow 2T(N/8) + N/4$

→ after kth step

$T(N) = 2^k T\left(\frac{N}{2^k}\right) + kN$

$k = \log_2 N$

$\Rightarrow T(N) = 2^{\log_2 N} T\left(\frac{N}{2^{\log_2 N}}\right) + \log_2 N \times N$

$= N \cdot T\left(\frac{N}{N}\right) + N\log_2 N$

$= NT(1) + N\log_2 N$

$T(N) = N + N\log N$

$TC \Rightarrow O(N\log N)$

$T(1) = 1$

$T\left(\frac{N}{2^k}\right) = 1$

$\frac{N}{2^k} = 1$

$\Rightarrow N = 2^k$

$\Rightarrow \log_2 N = \log_2 2^k$

$\Rightarrow k = \log_2 N$

⇒ call stack maxm depth → k step

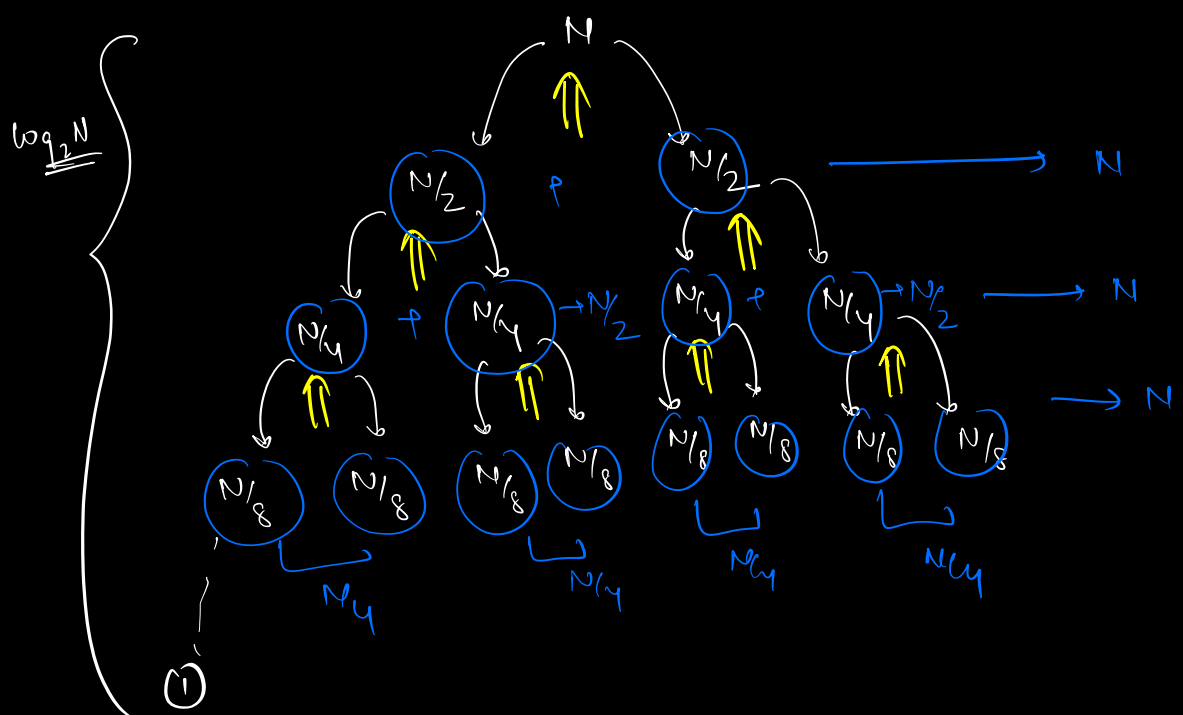
↓

$k = \log_2 N$

SC ⇒ $\log_2 N$ + N ⇒ **SC ⇒ O(N)**

↓ call stack

↓ temp array for merging

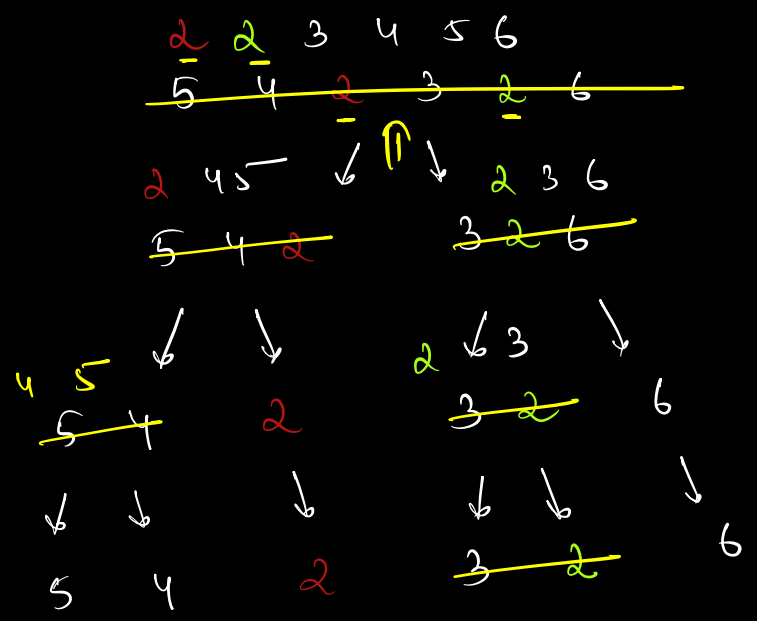


TC ⇒ no. of levels × time at each level

⇒ $\log_2 N \times N$

TC ⇒ $O(N \log N)$

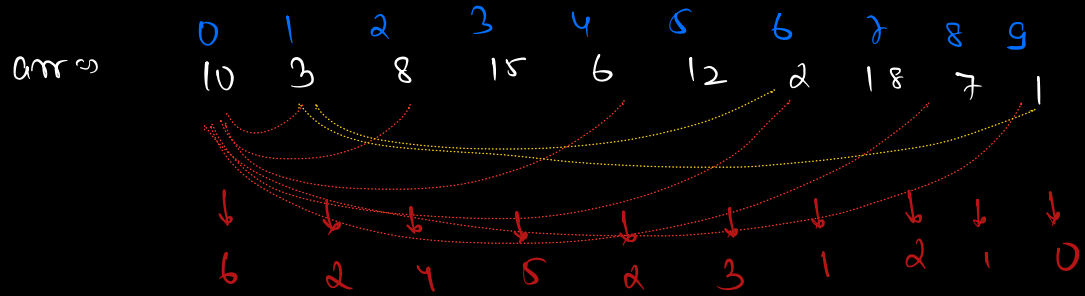
* inplace → X
* Stable → ✓



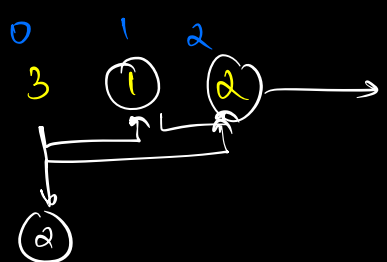
Google

(Inversion count)

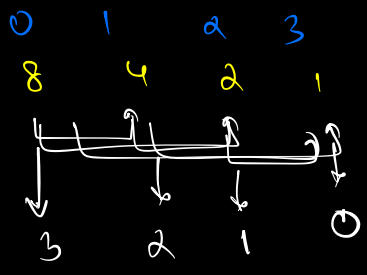
Q1. Given an array of N integers. find the count of pair (i, j), such that $i<j$ && $arr[i] > arr[j]$.



Quiz



Quiz



O/P = 6

Brute force

→ consider all possible pairs.

TC = O(N²)

Swapping ← { arr[i] > arr[j] (i<j) }

* bubble sort
* merge sort

merge sort

arr = [10 3 8 15 6 | 12 2 18 7 1]
(A) (B)

example → 10 6 5 8 3 2

TC → 5 + 3 + 2 + 2 + 1 + 0 = 13

A [10 6 5] | [8 3 2] B

A [2 1 0] B [2 1 0]

A+B+(A&B)
= 3+3+7
= 13

A&B → 3 + 2 + 2

arr ⇒ [10 3 8 15 6 | 12 2 18 7 1]

(A) (B)

Inversion count(arr) = IC(A) + IC(B) + IC(A2B) = 26

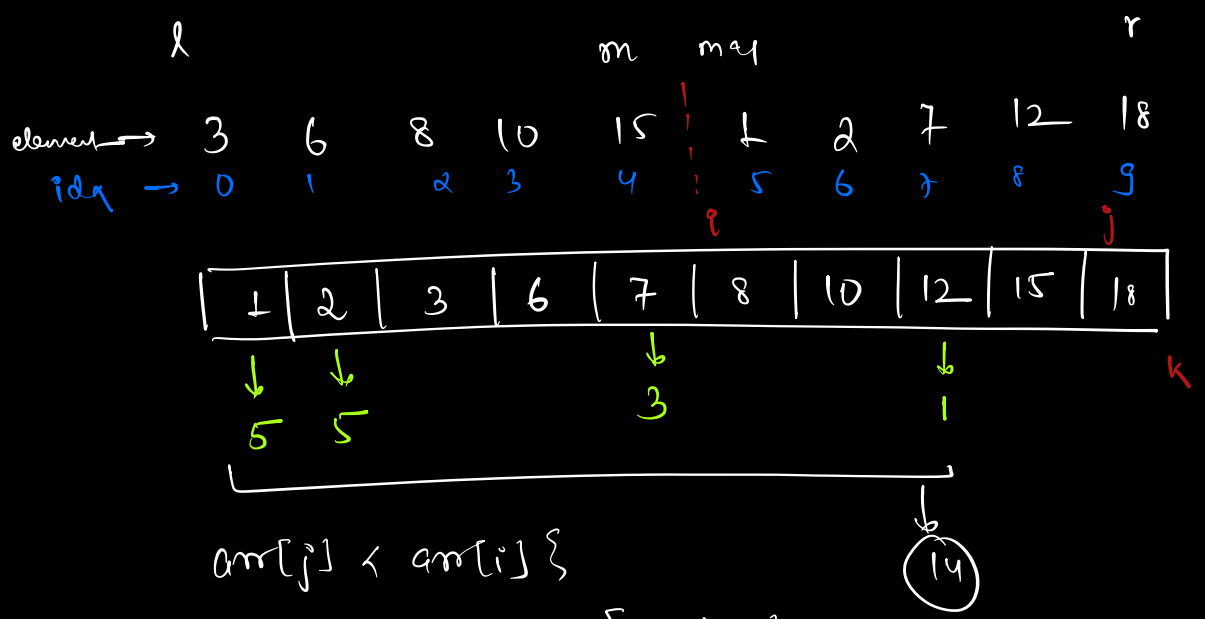
IC(A) ⇓ 5 IC(B) ⇓ 7 IC(A2B) ⇓ 14

10 3 8 15 6 | 12 2 18 7 1

Sort ⇒ 3 6 8 10 15 | 1 2 7 12 18

↑ 5 ↑ 5 ↑ 3 ↑ 1 ↑ 0

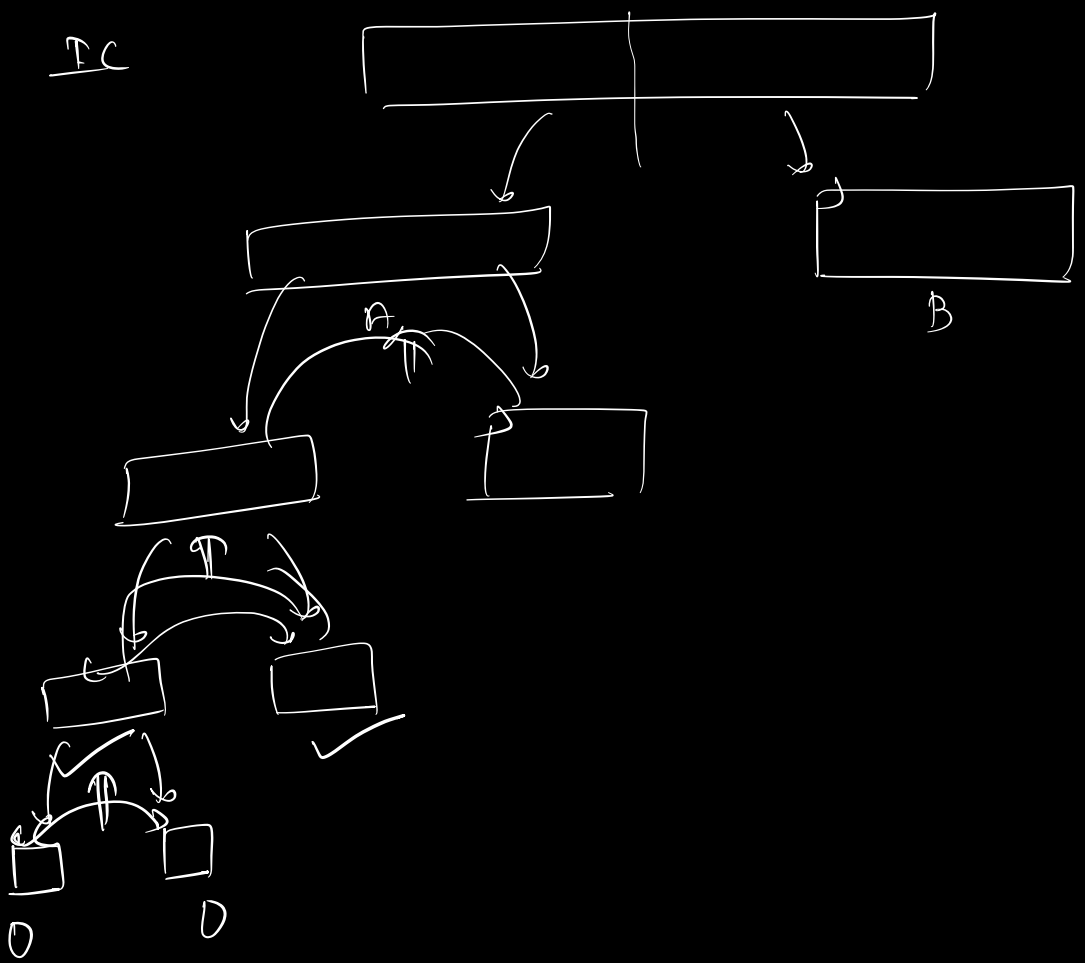
IC(A2B) = 14



arr[j] < arr[i] {

count = count + [m-i+1]

} else { i++ }


IC
B
D
D

```
int merge(a[], l, y, r){
    c[] = new arr[r-l+1]
    i=l; j=y; k=0;

    while( i<y && j<r ){
        if( a[i] <= a[j] ){
            c[k] = a[i]
            i++
            k++
        }
        else {
            count = count + (y-i)  or,
            c[k] = a[j]                count = count + (mid-i+1)
            j++
            k++
        }
    }
    while( i<y){
        c[k] = a[i]
        i++
        k++
    }
    while ( j<r) {
        c[k] = a[j]
        j++;
        k++;
    }
    return count;
}
```

← merge for mergesort

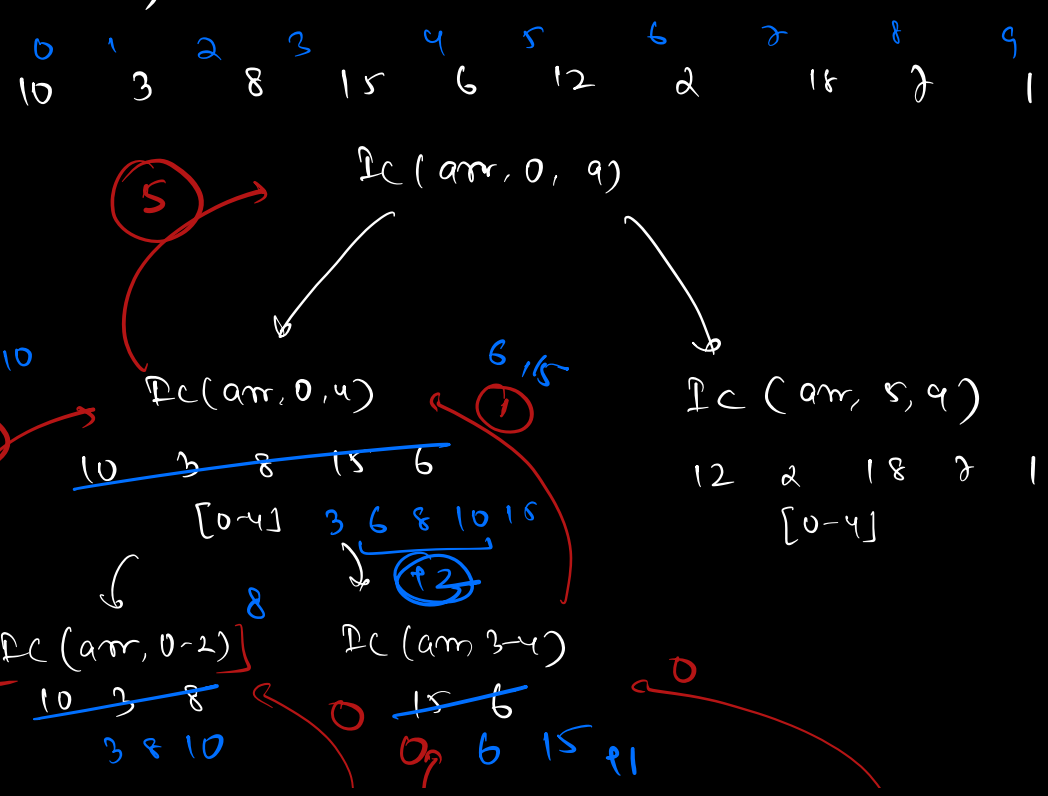
TC = O(r-l)

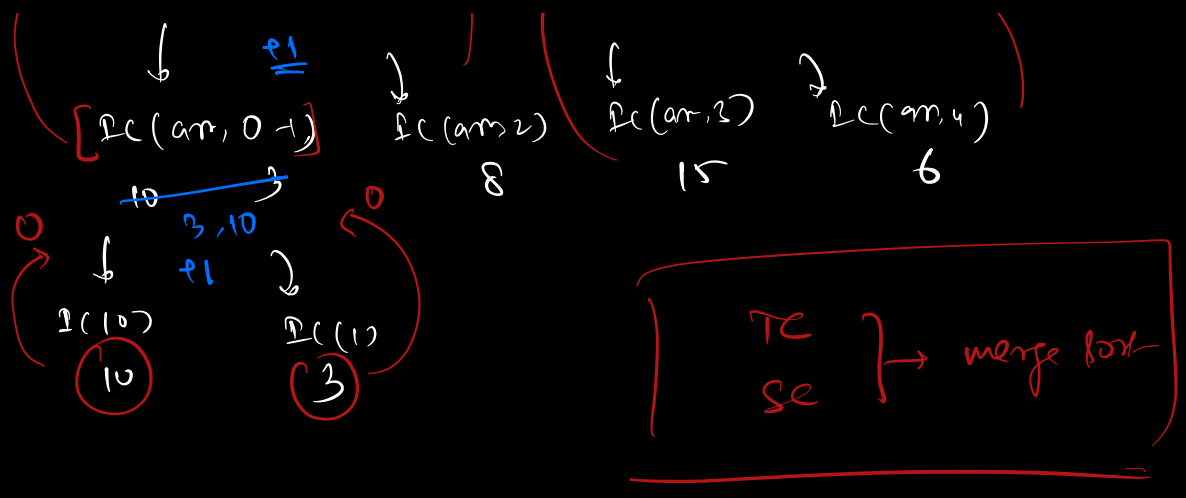
SC = O(r-l)

→ copy all elements from the newly sorted array to original array

Algm

```
int invCount(arr[], l, r) {
    if(l==r)
        return 0
    int mid = (l+r)/2
    int a = invCount(arr, l, mid)
    int b = invCount(arr, mid+1, r)
    int AB = merge(arr, l, mid+1, r)
    return a+b+(AB)
}
```

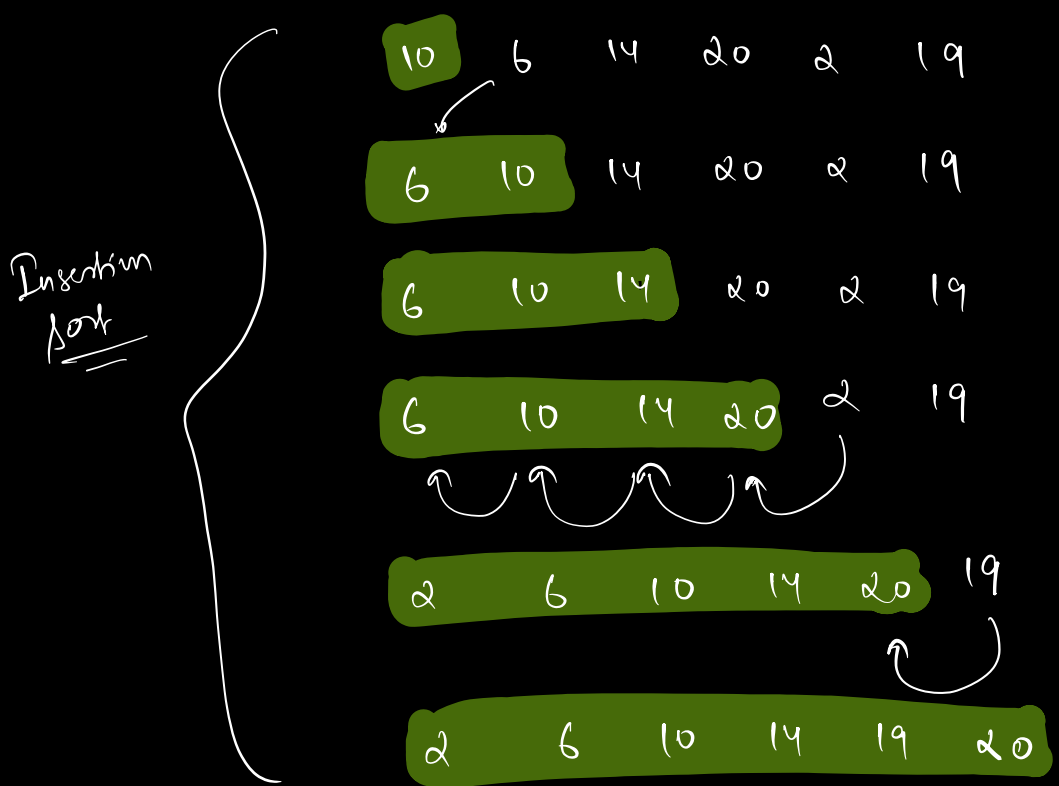




Insertion sort :

0	1	2	3	4	5
10	6	14	20	2	19

Insertion sort



```
for( i=1; i<N; i++){
    // sorted from 0 to i
    int j = i-1
    while( j>=0 && arr[j] > arr[j+1] ){
        swap( arr[j], arr[j+1] )
        j--
    }
}
```

TC $\Rightarrow O(N^2)$

SC $\Rightarrow O(1)$

* inplace → ✓
* stable → ✓

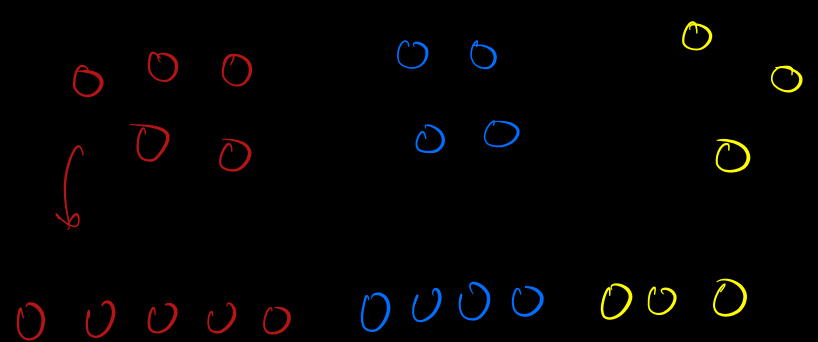
1 2 3 7 8 9 → sorted

if array is sorted

TC $\Rightarrow O(N)$

** nearly sorted

→ insertion sort



Count Sort

N = 15

[1 2 1 3 2 1 4 2 1 5 1 5 2 3 1]

no. of element = N

range = k.

$1 \le arr[i] \le 5$

5 - 1 + 1 = 5

	0	1	2	3	4	5
arr[6] ⇒	x	6	4	2	1	2

→ Create freq array, hashmap.

```
for(i=1; i<=5; i++){
    while(count[i]>0){
        print(i)
        count[i]--
    }
}
```

i	ith no.
1	6
2	4
3	2
4	1
5	2
5	15 = N

1 1 1 1 1 1 2 2 2 2 3 3 4 5 5

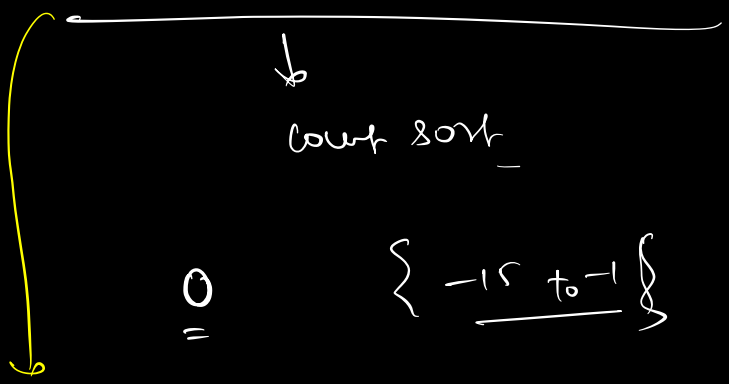
TC ⇒ O(N+k)

SC ⇒ O(k)

Q. [-15 -14 20 18 19 0 1 1 -2 -6 4 6 -6 -15
-12 18 19 -12 -7 1 3 2]

$-15 \leq arr[i] \leq 20$

Hint



Solve count sort with array

ex : 10^9 → integer.

N → length

$O(10^9 + N)$

$O(N \cdot \log N)$